

Travaux Pratiques 2

Miner un bloc en Python

Implémenter la Preuve de Travail (Proof of Work)

Abdoullah Ichou
EPSI — Auxerre

19 mai 2026

Durée : 1h30 **Groupes :** Individuel ou binôme **Prérequis :** Python 3.8+, bibliothèque standard uniquement

Objectifs pédagogiques :

- Implémenter une structure de bloc avec ses champs essentiels
- Comprendre et coder l'algorithme de Proof of Work (PoW)
- Mesurer expérimentalement l'impact de la difficulté sur le temps de minage
- Comprendre pourquoi le PoW rend la falsification computationnellement coûteuse

1 Mise en contexte

La **Preuve de Travail** (Proof of Work, PoW) est le mécanisme de consensus originel de Bitcoin. Il impose que pour qu'un bloc soit accepté par le réseau, son hash doit respecter une **condition de difficulté** : commencer par un certain nombre de zéros (en hexadécimal).

Pourquoi des zéros ? La seule façon de trouver un hash qui commence par d zéros est d'essayer de nombreuses valeurs. C'est ce qu'on appelle le **nonce** (number used once) : un compteur incrémenté jusqu'à ce que le hash satisfasse la condition.

Difficulté d	Condition à satisfaire
1	<code>hash.startswith("0")</code>
2	<code>hash.startswith("00")</code>
3	<code>hash.startswith("000")</code>
4	<code>hash.startswith("0000")</code>

La probabilité de trouver un hash valide par essai est de $\frac{1}{16^d}$. Pour $d = 4$, il faut en moyenne $16^4 = 65\,536$ tentatives.

2 Code de démarrage

Instruction : Le code ci-dessous est un squelette à compléter. Les parties à implémenter sont marquées # TODO. Ne modifiez pas les signatures des fonctions.

Fichier : `tp2_starter.py`

```

1 import hashlib
2 import time
3
4 # =====
5 # PARTIE 1 : Structure d'un Bloc
6 # =====
7
8 class Block:
9     """
10     Repr\`esente un bloc dans la blockchain.
11
12     Attributs :
13         index          -- position du bloc dans la cha\`{\i}ne
14         previous_hash  -- hash du bloc pr\`ec\`edent (maillon)
15         timestamp      -- horodatage de cr\`eation
16         data           -- donn\`ees/transactions du bloc
17         nonce          -- nombre d'essais pour le PoW
18         hash           -- empreinte cryptographique du bloc
19     """
20
21     def __init__(self, index, previous_hash, data):
22         self.index = index
23         self.previous_hash = previous_hash
24         self.timestamp = time.time()
25         self.data = data
26         self.nonce = 0
27         # TODO 1 : Calculer le hash initial du bloc
28         # Appel : self.hash = self.calculate_hash()
29         self.hash = None
30
31     def calculate_hash(self):
32         """
33         Calcule le SHA-256 du contenu du bloc.
34         La cha\`{\i}ne \`a hacher combine : index, previous_hash,
35         timestamp, data et nonce.
36         """
37         # TODO 2 : Construire la cha\`{\i}ne \`a hacher et retourner son
38         #           SHA-256
39         # Utilisez : hashlib.sha256(content.encode()).hexdigest()
40         # La cha\`{\i}ne doit inclure : str(self.index), self.
41         #                               str(self.timestamp), str(self.data),
42         #                               str(self.nonce)
43         pass
44
45     def __repr__(self):
46         return (f"Block(index={self.index}, nonce={self.nonce}, "
47               f"hash={self.hash[:12]}...)")
48
49 # =====
50 # PARTIE 2 : Proof of Work
51 # =====
52
53 def mine_block(block, difficulty=4):
54     """
55     Effectue la Preuve de Travail sur un bloc.

```

```

56     Incr\`emente block.nonce jusqu'\`a ce que block.hash
57     commence par 'difficulty' z\`eros.
58     Retourne le bloc min\`e (hash et nonce mis \`a jour).
59     """
60
61     prefix = "0" * difficulty
62
63     # TODO 3 : Impl\`ementer la boucle de minage
64     # Tant que block.hash ne commence pas par prefix :
65     #     - Incr\`emente block.nonce
66     #     - Recalculer block.hash
67     # Retourner le bloc une fois min\`e
68     pass
69
70
71 # =====
72 # PARTIE 3 : La Blockchain
73 # =====
74
75 class Blockchain:
76     """
77     G\`ere une liste cha\`ne de blocs avec validation.
78     """
79
80     def __init__(self, difficulty=4):
81         self.difficulty = difficulty
82         self.chain = []
83         # Cr\`eer le bloc de gen\`ese
84         self._create_genesis_block()
85
86     def _create_genesis_block(self):
87         """Cr\`ee et mine le premier bloc (index=0)."""
88         genesis = Block(0, "0" * 64, "Genesis Block")
89         # TODO 4 : Miner le bloc de gen\`ese et l'ajouter \`a self.chain
90         pass
91
92     def get_last_block(self):
93         """Retourne le dernier bloc de la cha\`ne."""
94         return self.chain[-1]
95
96     def add_block(self, data):
97         """
98         Cr\`ee un nouveau bloc, le mine et l'ajoute \`a la cha\`ne.
99         Retourne le temps de minage en secondes.
100         """
101         # TODO 5 : Cr\`eer un nouveau Block, le miner, l'ajouter \`a
102         # self.chain
103         # Retourner le temps \`ecoul\`e (utilisez time.time() avant et
104         # apr\`es)
105         pass
106
107     def is_valid(self):
108         """
109         V\`erifie l'int\`egrit\`e compl\`ete de la blockchain.
110         Pour chaque bloc (sauf le gen\`ese) :
111         - Son hash enregistr\`e correspond \`a son hash recalcul\`e
112         - Son previous_hash correspond au hash du bloc pr\`ec\`edent
113         - Son hash satisfait la condition de difficult\`e

```

```

112     """
113     # TODO 6 : Implémenter la validation compléte
114     pass
115
116
117 # =====
118 # PARTIE 4 : Expérimentation
119 # =====
120
121 if __name__ == "__main__":
122     print("=== Test de minage avec différentes difficultés ===\n")
123
124     for difficulty in [1, 2, 3, 4]:
125         bc = Blockchain(difficulty=difficulty)
126         start = time.time()
127         elapsed = bc.add_block("Transaction: Alice -> Bob : 10 BTC")
128         duration = time.time() - start
129
130         last = bc.get_last_block()
131         print(f"Difficulté {difficulty} | Nonce = {last.nonce:>8} | "
132               f"Hash = {last.hash[:20]}... | Temps = {duration:.4f}s")
133
134     print("\n=== Validation de la chaîne ===")
135     bc = Blockchain(difficulty=3)
136     bc.add_block("Alice -> Bob : 5 BTC")
137     bc.add_block("Bob -> Charlie : 2 BTC")
138
139     print("Blockchain valide :", bc.is_valid())
140
141     # TODO 7 (bonus) : Tenter de falsifier bc.chain[1].data et relancer
142     # is_valid()
143     # Que constatez-vous ?

```

3 Exercices

Exercice 1 — Compléter le squelette [TODO 1 à 6]

Complétez les parties # TODO dans l'ordre numérique. Testez après chaque étape. À la fin, exécutez `python tp2_starter.py` et observez les résultats.

Exercice 2 — Mesure expérimentale de la difficulté

Modifiez le code pour tester les difficultés 1, 2, 3, 4 et 5. Remplissez le tableau :

Difficult�	Nonce moyen	Temps (s)	Ratio vs d=1
1			1
2			
3			
4			
5			

Question : Quel est le facteur th orique entre deux difficult s cons cutes ? Vos mesures confirment-elles la th orie ?

Exercice 3 — Falsification et immuabilit  [TODO 7]

1. Cr ez une blockchain de 3 blocs (difficult  3).
2. Modifiez directement `bc.chain[1].data = "Alice -> Mallory : 50 BTC"`.
3. Appelez `bc.is_valid()`. Que se passe-t-il ?
4. **Bonus avanc  :** Recalculez `bc.chain[1].hash` apr s la modification. La validation  choue-t-elle toujours ? Pourquoi ?

Exercice 4 (Bonus) — Repr sentation graphique

En utilisant `matplotlib`, tracez un graphique du temps de minage en fonction de la difficult . Commentez l'allure de la courbe (lin aire ? exponentielle ?).